- <technique>
  - <AI/Artificial Intelligence>
    인간지 지능을 모방하는 소프트웨어를 통칭하는 추상적 개념
    - <machine learning>
      데이터로부터 패턴을 자동 학습하는 AI 방법론
      - [지도 학습]
      [Supervised learning]
      입력과 정답을 제공하는 방식
      - [비지도 학습]
      [Unsupervised learning]
      정답 없이, 데이터 스스로 군집(clusters) 패턴 학습
      - [강화 학습]
      [Reinforcement learning]
      보상(reward)을 통해 최대화되는 행동 학습
      - <classical ML>
      사람이 데이터의 특징을 정의하고, 특징을 근거로 예측을 학습하는 ML
      - <deep learning>
      사람이 특징 추출하지 않고 학습하는 ML
      - <transformer>
      문맥 데이터 간 관계 파악에 특화되는 아키텍처
      - <Language Model>
      텍스트 데이터로 언어 패턴을 학습한 모델
      - [reasoning model]
      중간 추론을 학습하여 출력하고 이전 추론이 답변에 학습되는 모델
      - <Vision Model>
      이미지 데이터로 시각 패턴을 학습한 모델
      - <Audio Model>
      오디오 데이터로 소리 패턴을 학습한 모델
      - <Retrieval-Augmented Generation>
      외부 지식베이 검색 정보를 검색하여, LLM 답변에 보충시키는 기법
      - [retrieval method]
      - [vector search]
      텍스트 임베딩으로 의미적 유사도 검색
      - [keyword search]
      단어 일치 및 용어 빈도로 검색
      - [graph search]
      graph DB 검색
      - [SQL search]
      RDBMS 테이블 쿼리 기반 정형 데이터 검색
      - [web search]
      - <AI agent>
      계획을 실행하기 위해 자율적으로 반복하고 필요하는 외부 도구를 사용하는 AI 구성 방식
      - <agent architecture pattern>
      - [ReAct (Reasoning+Acting)]
      thought-action-observation 루프 반복
      - [plan-and-execute]
      전체 계획 수립 → 단계별 실행 분리된 구조
      - [tool-use]
      LLM이 외부 함수·API를 호출하는 구조
      - [memory]
      대화·세션 간 지속되는 정보 저장 방법
      - [reflection]
      실행 결과를 스스로 평가하고 개선하는 자기 수정 루프
    - <knowledge-based>
      사람이 해석을 수동으로 정의하는 AI 방법론
  - <Database Management System>
    데이터베이스를 관리하는 소프트웨어
    - [database]
    과업 시스템의 중속성, 중복성 문제를 해결하기 위해 고안된 데이터 저장 방식.
    [Database] 데이터 저장 방식 자체를 가리키는 개념
    (구현하는 값만 데이터가 아닌 비일정 중복 데이터는 문제)
    - <Relational Database>
      데이터를 정형화하여 테이블 구조로 구분하고 테이블 간 관계를 정의하여 사용한다.
      - [transaction]
      DB process unit
      -Atomicity: commit || rollback
      -Consistency: constraint
      -Isolation: race condition prevent
      -Durability: logging
      - [trigger]
      데이터, 상태가 변경되어 특 이벤트 발생시 수행되는 프로젝션
      - <DDL>
      [Structured Query Language]
      relational database language
      - <DML>
      (Data Define Language)
      - <DDL-keyword>
      [CREATE] 생성 구조 생성
      [CREATE FieldType fieldname ...]
      [ALTER] 필드 구조 변경
      [ALTER fieldType fieldname ADD | ALTER | DROP COLUMN]
      [DROP] 필드 구조 삭제
      [DROP fieldType fieldname CASCADE | RESTRICT]
      - <Database structure>
      SCHEMA, DOMAIN, VIEW, INDEX, TABLE
      - [SCHEMA]
      전반적인 구조(인덱스, 뷰, 테이블, 도메인) 그룹화 장서
      - [DOMAIN]-AS type CHECK(제약조건)
      제약조건을 사용하여 사용자 정의 속성 타입 생성
      - [INDEX]-ON tablename(attribute)
      binary search 등의 탐색을 위해 적정한 컬럼을 정할 경우에는 별도 인덱 구조
      - [VIEW]-AS SELECT - FROM ...
      - SELECT 문으로 조회하여 구축한 가상 테이블
      - [TABLE]-(type attributename, ..., CHECK(제약조건), ...)
      - mysqlexecute -u root -p -D dbname < filename.txt
      - mysqldump -u root -p dbname > filename.txt
      - key: 기본키, 구분가능, 고유한 값들과 이루어진 속성값
      - [binary search]
      정렬된 데이터에서 중간값과 비교하여 대상 절반씩 제외하는 탐색 방식. 시간 복잡도 O(log n)
      - <DML>
      (Data Manipulation Languages)
      - <DML-keyword>
      [SELECT] 테이블 내 레코드 검색(추출)
      [INSERT] 테이블 내 레코드 삽입
      [INSERT INTO 테이블(컬럼,...) VALUES(속성값2, ...)]
      [UPDATE] 테이블 내 특정 속성 변경
      [UPDATE 테이블 SET 속성1=변경값 WHERE 조건가]
      [DELETE] 테이블 내 레코드 삭제
      [DELETE FROM 테이블 WHERE 조건가]
      - <SELECT>
      - SELECT [DISTINCT] [table_name.]{ attrname1, ‖ group function,...
      FROM table [AS rename],
      - [조건]
      [WHERE [NOT] condition]
      [WHERE condition AND | OR condition]
      [WHERE attribute [NOT] IN (값,값,값)]
      [WHERE attribute BETWEEN condition AND condition]
      [WHERE condition IS [NOT] NULL]
      - [정렬]
      [ORDER BY attribute ASC(오름차순) | DESC(내림차순), ...]
      - [그룹화 집합 정의]
      [GROUP BY attribute, ...HAVING condition]
      - [집합 결합(서로 붙이 주기)]
      [UNION(합집합) | UNION ALL(교집합) | MINUS(차집합)]
      INTERSECT(교집합) | EXCEPT(차집합)]
      SELECT attribute, ..FROM table_1
      - [조인 방법]
      [INTER JOIN(교집합)]
      [OUTER JOIN(합집합)]
      LEFT OUTER JOIN(왼쪽기준합집합)]
      RIGHT OUTER JOIN(오른쪽기준합집합)]
      CROSS JOIN(카테시안)
      table ON condition...]
      - <aggregate function>
      [COUNT(attribute)] 개수
      [SUM(attribute)] 합
      [AVG(attribute)] 평균
      [MAX(attribute)] 최대값
      [MIN(attribute)] 최소값
      [STDDEV(attribute)] 표준편차
      [VARIANCE(attribute)] 분산
      [ROLLUP(attribute, ...] 소계 추가
      [CUBE(attribute, ...] 총계 소계
      - <like-wildcard character>
      [%] 모든 문자
      [_] 단일 문자
      - <DCL>
      (Data Control Language)
      - <DCL-keyword>
      [COMMIT] 영향이 수행 결과 확정
      [ROLLBACK] 영향이 수행 이전으로 복귀
      [SAVEPOINT] ROLLBACK 위치 지정
      [GRANT] 영향에 권한 부여
      [GRANT 권한명 ON 테이블명 TO 대상 (WITH GRANT OPTION)]
      [REVOKE] 영향에 권한 회수
      [REVOKE (GRANT OPTION FOR) 권한명 ON 테이블명 FROM 대상]
  - <planning>
    - 무엇을 만들 것인가
    [Product Requirements Document]
    - 초기 요구사항 및 기능 정의
  - <design>
    - 어떻게 만들 것인가
    - <UI/UX design>
    - 구성 요소, 인터 조표 정의
    [prototype]
    - wire frame -> mockup -> prototype
    - <software design>
    [flow chart]
    - 소프트웨어가 해결한 목표 조표 정의
    [Entity-Relationship Diagram]
    - <hardware design>
    [system architecture]
    - 하드웨어, 네트워크 요할 인해 구조 정의
  - <development>
    - <programing paradigm>
    - 어떻게 사용할 것인가
    - <declarative programming languages>
    [Declarative]
    ("how" is implemented by system.)
    - <imperative programming languages>
    "what" + "how"
    - <Object-Oriented Programming>
    - 사물
    - <Class>
    - <attribute>
    - <method>
    - <Function>
    - <Function>
    - 기능 및 정의 단위를 의미 하는 용어
    - 수학 함수 f(x)=x²+3x+5 처럼 입력을 받아 결과를 돌려주는 블록
    - 프로그램에서 특정 작업을 수행하는 명령 묶음
    - 이름을 붙여 재사용 가능하게 만들어 놓은 것
    - 입력(parameters) 을 받아 처리하고 결과(return value)를 돌려주는 구조
    - parameters: 함수 정의 시 괄호 안에 적는 변수 이름들
    - return value: 함수가 호출한 곳으로 되돌려주는 값
    - 함수는 한 번 정의하면 여러 번 재사용 가능
    - <abstraction>
    - 내용들을 추출하여 일반화
    - [class]
    - 연구에 필요에 대한 추상화
    - constructor: 객체 생성 및 초기화 수단
    - [interface(OOP)]
    - 클래스에 대한 추상화
    - [object]
    - 구체화
    - <encapsulation>
    - 인터페이스만 밖에 드러내고 내부 구현을 감춘다.
    - 변수, 함수, 클래스, 인터페이스가 내부 접근 범위 설정
    - [access modifier]
    - -default(package private): 같은 패키지 안에서만 접근 허용
    - -public(or 생략): 모든 곳에서 접근 가능
    - -protected: 클래스 내부에서만 접근 허용
    - -protected(pkg): 클래스 내부와 같은 패키지에서 접근 허용
    - <inheritance>
    - 새로운 클래스가 기존 클래스의 속성과 메서드를 물려받는 것.
    - 코드 재사용과 계층 구조를 만들기 위한 핵심 개념.
    - [extending]
    - 상속받은 클래스는 부모 클래스의 기능을 그대로 가지고 시작하고, 필요한 것만 추가하거나 바꿔서 사용할 수 있다.
    - [overriding]
    - 상속받은 메소드는 재정의할 수 있다.
    - [polymorphism]
    - 다형성이 가능할 수 있는 상황
    - [overloading]
    - 같은 클래스 안에서는 매개변수 목록이 다르다면 같은 이름의 메소드를 여러 개 가질 수 있다.
    - [overriding]
    - 상속받은 메소드는 재정의할 수 있다.
    - [generic]
    - 제개변수 사용법을 일반화
    - <SOLID principle>
    - 객체지향의 대원칙
    - [Single Responsibility Principle(SRP)]
    - 하나의 클래스는 하나의 책임만 가져야 한다.
    - [Open Close Principle(OCP)]
    - 확장에는 열려있고, 변경에는 닫혀있어야 한다.
    - [Liskov Substitution Principle(LSP)]
    - 자식 클래스는 언제나 기반 클래스의 자리를 대체할 수 있어야 한다.
    - [overriding 시 특히 중요 확장할 수 있어야, 전문화되는 것만이.]
    - [Interface Segregation Principle(ISP)]
    - 거대 하나 인터페이스보다 작고 목적 있는 인터페이스 여러 개가 낫다.
    - [Dependency Inversion Principle(DIP)]
    - 추상에에 의존하라. 구체에에 의존하지는 말라.
    - <Procedural Programing>
    - 절차
    - <sequential programing>
    - 순차적 흐름 : 명령문이 위에서 아래로 한 줄씩 실행되는 가장 기본적인 흐름.
    - <function>
    - 함수에 정의된 명령문들을 이름만 불러 호출하는 것.
    - <arguments>: 함수에 넘겨주는 데이터
    - <return value>: 함수가 넘겨주는 데이터
    - <structure>
    - 변수에 대한 추상화
    - <procedure>
    - 과업에 개념할 것인가
    - <control statements>
    - [conditional statement]
    - <if(else)
    - <switch(case)
    - [iteration statement]
    - <for
    - <while
    - <do-while
    - [branch statement]
    - <return: 함수를 종료하고 호출한 곳으로 되돌아감.
    - <break: 현재 반복문을 즉시 멈춤.
    - <continue: 현재 반복만 중단하고 다음 반복으로 넘어감.
    - <try-catch>
    - [try] 실행할 코드
    - [catch] 오류가 모든 발생 시 실행할 코드
    - <operator>
    - [increment & decrement]
    - ++i (i 증가) --i (i 감소)
    - [arithmetic]
    - *(곱셈),/(나눗셈),%(나머지)
    - +(더하기), -(빼기)
    - [relational]
    - <(작다),>(크다),<=(이하),>=(이상),==(같다),!=(다르다)
    - [logical]
    - !(NOT) &&(AND) ||(OR)
    - [bit]
    - &(and) ^(xor) |(or) ~(not)
    - <(데이터 이동)>(데이터 분리)
    - (floating point): 부동점수 (정수가 아닌 수)
    - [ternary]
    - ? (true) ‖ false)
    - [assignment]
    - =(두 수 대입)
    - <algorithm>
    - 문제해결 절차
    - [logic]
    - 최선 문제
    - <search algorithm>
    - 자 방법 값에서 원하는 값찾기
    - [binary search]
    - 정렬된 데이터에서 중간값과 비교하여 절반씩 제외하는 탐색 방식. 시간 복잡도 O(log n)
    - [Breadth-First Search]
    - 큐(FIFO) 한개에서 모든 정점을 레벨 단위로 탐색
    - [Minimum Spanning Tree]
    - [shortest path]
    - <sorting algorithm>
    - [insertion sort] (O(N²))
    - 정렬되지 않은 데이터 한 개씩 꺼내 기존 정렬된 부분에 삽입
    - [selection sort] (O(N²))
    - 전체에서 최소값을 찾아 맨 앞에 교체 하는 과정 반복
    - [bubble sort] (O(N²))
    - 인접한 두개 씩 비교해 값이 잘못 되어 있으면 교환하는 과정 반복
    - [quick sort] (O(NlogN)~O(N²))
    - 피벗 값 < 작은 값(pivot) < 큰 값으로 분할, 반복
    - [heap sort] (O(NlogN))
    - 데이터를 최대 힙(max heap) 구조로 만들어 루트(최대값)를 꺼내고 힙 구조를 복원하는 과정을 반복하여 정렬
    - [merge sort] (O(NlogN))
    - 리스트 요소가 1개 될때까지 반복해 이분할 결합된 역순으로 재배열하여 정렬
    - <recursive algorithm>
    - 기본 케이스(종료 조건)와 자기 자신을 재호출하는 재귀 메서드를 단계마다 작은 문제를 처리한다.
    - <graph algorithm>
    - [Depth-First Search]
    - -전위 순회
    - -중위 순회
    - -후위 순회
    - [Breadth-First Search]
    - -루트부터 인접해서 모든정점을 레벨단위로 탐색
    - [shortest path]
    - <string search algorithm>
    - [brute force]
    - 단어는 한 단어 씩 일치하는 위해 탐색
    - [Boyer Moore]
    - 문자열 내 패턴을 찾는 효율적인 알고리즘
    - 뒤에서부터 비교하고 불일치 발생 시 패턴을 오른쪽으로 최대한 멀리 이동하여 불필요한 비교를 줄이는 방식. 평균 시간 복잡도 O(n/m) 수준
    - <operand>
    - 어떻게 사용할 것인가
    - <structure>
    - <array>
    - <continuous [index] of same primitive type>
    - [Global Ivo]
    - [node] = base + next_node_address
    - <embedded structure>
    - [Ivo] 수식에 이러다 문제는 문제는 해결하기 어렵
    - [vo] 수식에 이러다 문제는 문제는 해결하기 쉽 어렵
    - [Adjacency]: Ivo, ->Ivo,
    - [Stack]: 마지막 들어온 것이 먼저 나감 (LIFO)
    - [Queue]: 먼저 들어온 것이 먼저 나감 (FIFO)
    - [Deque]: 양쪽 모두 넣고 빼기 가능
    - [Linked List]: 각 노드가 다음 노드를 가리키는 방식
    - [array]: Vertex + Edge
    - -tree: node + data + parent_address + children_address
    - -binary tree: limit child_addresses to a maximum of 2
    - <type_management>
    - [static typing]: compile-time typing
    - [dynamic typing]: run-time typing
    - 컴파일 단계에 문법 검사가 모두 끝나고 프로그램이 실행되기 전에 모든 변수의 타입이 확정되어 있는 방식. 실행 중에 타입 때문에 갑자기 에러가 나는 일이 거의 없음.
    - 변수 이름만 있고 타입은 없음. 실행 중에 값이 들어오면 그때 타입을 결정하거나 바꿔도 됨. 유연하지만 실행 중 에러 발생 가능성이 높음.
    - <memory allocation>
    - [static allocation]: compile-time allocation
    - [dynamic allocation]: run-time allocation
    - <lifetime>
    - [local lifetime]: lifetime in call stack
    - [global lifetime]: lifetime in run-time
    - <scope>
    - [static scope (lexical)]
    - (block scope): 변수는 선언된 블록을 그 밖에 벗어나면 유효하다.
    - (function scope): 변수는 선언된 함수에 그 밖에 벗어나면 유효하다.
    - [dynamic scope]
    - 호출되어 스택에 남아 있는 동안 유효하다.
    - <memory management>
    - <memory>
    - <process_state>
    - [new state]: 프로그램이 메모리에 올라오고 PCB를 할당받은 상태
    - [ready state]: CPU 스케줄러가 지정한 시간에 프로세서 상태 대기 중인 상태
    - [running state]: 프로세스가 CPU를 점유하여 실행하는 상태
    - [waiting state]: 실행 상태에 프로세스가 입출력 요청에 의한 이벤트 상태
    - [terminated]: 프로세스가 종료되고 할당된 메모리를 반납하는 상태
    - <lifetime>
    - [Lifetime]
    - [Process Lifetime]: 프로세스 생성부터 종료까지 유지되는 가장 넓은 범위. 프로세스 전체 동안 메모리가 할당되어 있음.
    - [Thread Lifetime]: 스레드 하나가 실행되는 동안만 유지되는 범위. 스레드 종료 시 해당 스레드 전용 메모리는 사라짐.
    - [Function Lifetime]: 함수 호출 시 생성되고 함수 종료 시 사라지는 지역 변수의 생명주기.
    - [Block Lifetime]: 중괄호 { } 안에 선언된 변수는 블록을 벗어나는 순간 메모리에서 사라짐.
    - <process_management>
    - [Process Control Block (PCB)]
    - 운영체제가 프로세스를 관리하기 위해 유지하는 데이터 구조. 프로세스의 모든 정보를 담고 있음.
    - [Context Switching]
    - 한 프로세스가 CPU를 내놓고 다른 프로세스가 CPU를 차지하는 과정. 이때 PCB의 정보를 저장하고 불러오는 작업이 발생함.
    - [Process Scheduling]
    - 운영체제가 어떤 프로세스에게 CPU를 줄지 결정하는 과정. 스케줄러가 PCB 정보를 참고하여 결정함.
    - [Process Table]
    - 모든 프로세스의 PCB를 모아 놓은 커널 내부의 표. 운영체제가 프로세스를 관리할 때 가장 먼저 참조하는 데이터.
    - <process_synchronization>
    - [Critical Section]
    - 여러 프로세스가 동시에 접근하면 안 되는 공유 자원을 다루는 코드 구간.
    - [Mutual Exclusion]
    - 한 프로세스가 공유 자원을 사용하는 동안 다른 프로세스는 접근하지 못하도록 하는 것.
    - [Deadlock]
    - 두 프로세스가 서로 상대방이 가진 자원을 기다리며 영원히 멈추는 상태.
    - [Semaphore]
    - 자원 개수를 세는 신호등. 프로세스가 자원을 얻으면 숫자를 줄이고, 반납하면 숫자를 늘림.
    - [Mutex]
    - 한 번에 한 명만 들어갈 수 있는 화장실 문 잠금장치. 소유한 프로세스만 잠금을 풀 수 있음.
    - [Monitor]
    - 세마포어와 뮤텍스를 좀 더 안전하게 감싸는 상위 개념.
    - [Spinlock]
    - 자원이 풀릴 때까지 계속 while 문을 돌면서 기다리는 방식. 컨텍스트 스위칭 비용이 없지만 CPU 자원을 낭비함.
    - [Busy Waiting]
    - 자원이 풀릴 때까지 계속 while 문을 돌면서 기다리는 방식. 컨텍스트 스위칭 비용이 없지만 CPU 자원을 낭비함.
    - [Producer-Consumer Problem]
    - 생산자가 데이터를 만들고 소비자가 이를 가져가는 구조. 버퍼가 가득 차거나 비었을 때 동기화가 필요함.
    - [Reader-Writer Problem]
    - 읽는 사람은 여럿이 동시에 볼 수 있지만, 쓰는 사람은 혼자 와야 하는 문제. 데이터 일관성을 위해 중요함.
    - [Dining Philosophers Problem]
    - 식사하는 철학자들이 젓가락을 두 개씩 집어야 하는데, 모두 한 쪽씩 집어 들면 아무도 먹을 수 없는 교착 상태 문제.
    - [Starvation]
    - 누군가는 계속 자원을 못 얻어 굶어 죽는 상태. 공정한 스케줄링이 필요함.
    - [Priority Inversion]
    - 낮은 우선순위 프로세스가 자원을 쥐고 있어 높은 우선순위 프로세스가 멈추는 현상. 시스템 전체 성능 저하 유발.
    - [Priority Ceiling]
    - 자원을 쓰는 프로세스에 미리 최고 우선순위를 부여하여 중간 우선순위 프로세스가 끼어들지 못하게 하는 방법.
    - [Priority Inheritance]
    - 높은 우선순위 프로세스가 기다리면 자원 쥔 프로세스에게 잠시 그 우선순위를 빌려주는 방법.
    - <I/O system>
    - <driver>
    - software-hardware interface
    - <CPU>
    - <CPU instruction cycle>
    - [instruction fetch]
    - ↓
    - [opcode decoding]
    - ↓
    - [operand fetch]
    - ↓
    - [instruction execute]
    - ↓
    - [result store]
    - <ALU>
    - -alu/calculate
    - <control unit>
    - -decoding
    - <register>
    - [Program Counter] next opcode address
    - [Instruction Register] opcode
    - [Memory Buffer Register] value operand
    - [Memory Address Register] address operand
    - [general purpose register] operand
    - [flag register] additional result state
    - [stack pointer] topmost stack address
    - [base register] base address
    - <CPU access>
    - Register > L1 Cache > L2 Cache > L3 Cache
    - [memory] dozens-ms
    - [drive] hundreds-ms
    - <hardware>
    - <memory>
    - <Random Access Memory>
    - 램은 전기를 끄면 모든 데이터가 사라지는 휘발성 메모리
    - <Read-Only Memory>
    - 전원이 꺼져도 데이터가 사라지지 않는 비휘발성 메모리. 주로 펌웨어나 시스템 부팅 정보를 저장하는 데 사용됨.
    - <I/O>
    - <drive>
    - 사용량 데이터 저장을 위한 장치
    - [LAN card]
    - 유선 NIC
    - [WIFI card]
    - 무선 NIC
    - <Network Interface Card>
    - <Operating System>
    - <memory system>
    - memory layout
    - [physical address]: absolute address
    - [logical/virtual address]: relative address
    - <memory trade-off>
    - [virtual memory]: ssd drive as memory (swap)
    - <memory partition>
    - [paging] 고정 크기 페이지 단위로 물리적 메모리를 분할하는 방식
    - [segmentation] 가변 크기 세그먼트 단위로 메모리를 분할하는 방식
    - <thrashing>
    - 페이지 교체가 CPU 연산보다 많아지고 CPU가 다른 일을 못하는 현상
    - <process memory layout>
    - [kernel area]: OS code(mapping tabs. ..)
    - [code area] opcode + operand address
    - [data area]: static allocation, global lifetime
    - [heap area]: dynamic allocation, global lifetime
    - [stack area]: function frame, local lifetime
    - <file system>
    - drive layout
    - [block]
    - -smallest file data unit
    - [file]
    - -file name
    - -file attribute
    - [directory file/dirent]
    - -file address table
    - <drive partition method>
    - [FAT(File Allocation Table) file system]
    - store block addresses in linked list structure
    - [unix file system]
    - store block data and addresses in tree structure
    - <I/O system>
    - <software>
    - <Software Development Life Cycle>
    - <software>
    - finally integrated component
    - [code] -operand(logic) + operand(value, address)
    - [data] -value + type
    - [file] code in drive
    - [program] executable file
    - [process] program in memory
    - [module] functional unit code
    - [component] service unit code
    - [package] group of unit codes
    - [library] reusable detail package
    - [framework] reusable structure package
    - <API>
    - (Application Programming Interface)
    - software-software interface
    - [Service Provide Interface]
    - host-defined interface for plugins
    - <computer>
    - hardware(executor) + software(instruction)
    - <computer>
    - <networks>
    - [gateway]
    - network-network interface
    - <protocol>
    - 통신 규약
    - [OSI 7계층]
    - 프로토콜 네트워크 작동 구조
    - [POU(Protocol Data Unit)]
    - 계층별 캡슐화 단위
    - <application layer protocol>
    - [HTTP(HyperText Transfer Protocol)]
    - 인터넷을 위하여이 웹상 자료에 프로토콜
    - [URI(Uniform Resource Identifier) =URL]
    - [DNS(Domain Name System)]
    - ip 자장 서비스
    - [SNTP]
    - [NTP] ip 컴퓨 서버에 ip 값을 서버에 서비스
    - [SSH(Secure Shell)]
    - -telnet 후속 서비스 (암호화된) 원격 접속 서비스
    - [SMTP]
    - -e-mail 서비스
    - [FTP]
    - -file 전송 서비스
    - <transport layer protocol>
    - [TCP(Transmission Control Protocol)]
    - -connectivity, reliability
    - [P2P, multiplayer ...]
    - [UDP(User Datagram Protocol)]
    - -non-connectivity, non-reliability
    - [broadcast, streaming service ...]
    - <network layer protocol>
    - [IP(Internet Protocol)]
    - -ip address, packet
    - [ICMP, ARP, IGMP, ...]
    - [RIP (I GRP ..., BGP ...]
    - -route selection algorithm
    - <ethernet>
    - [IEEE 802.3 protocol]
    - -define encapsulation
    - [CSMA/CD protocol]
    - -separate collision domain in wire network
    - <wireless network
    - -IEEE 802.11
    - -CSMA/CA
    - <node>
    - [DTE(Data Terminal Equipment)]
    - 동기화 및 흐름 제어 기능
    - -client -requester
    - -server -responder
    - [DCE(Data Circuit-terminating Equipment)]
    - 동기화를 위한 클럭 신호 공급
    - -intermediary device
    - <application layer>
    - -Application layer>
    - [L7(application layer)]
    - -L6(presentation layer)
    - -L5(session layer)
    - POU: message (byte unit)
    - 사용자가는 어디로 연결되는지 고민하고 개발하여야 하는 레이어
    - <socket>
    - -network API
    - <4.transport layer>
    - POU: segment(TCP), datagram(UDP)
    - [port number]
    - process identifier
    - [connectivity]
    - -check for communication with 3-way handshake
    - [reliability]
    - -ensures no data is missing
    - <3.network layer>
    - POU: packet
    - [IP address]
    - -LAN identifier
    - [intermediary device]
    - -router: separate 2 connect [broadcast domain]s
    - <2.data link layer>
    - POU: frame
    - [MAC address]
    - -node identifier
    - [mac access]
    - -CSP: [1:1]
    - -multipoint: [1:n]
    - [intermediary device]
    - -switch: separate 2 connect [collision domain]s
    - -VLAN: separate broadcast domain through a switch]
    - <link>
    - -node-node interface
    - <1.physical layer>
    - POU: bits
    - [physical transmission line]
    - -wire(metal cable)
    - -wireless
    - [line connection]
    - [network topology]
    - -mesh: 모든 노드가 서로를 연결
    - -star: 모든 노드가 중앙 허브(hub)를 연결
    - -bus: 하나 노드가 하나의 회선에 연결
    - -tree: 여러 star가 bus로 연결
    - -ring: 연결된 노드 마지막 서로를 연결
    - [intermediary device]
    - -repeater: signal amplification
    - -hub: connection amplification
    - -modem: digital [] analog signal modulation
    - <cloud architecture>
    - [on-premise]
    - non cloud
    - [Infrastructure as a Service]
    - = virtualization server, storage, network
    - [Platform as a Service]
    - runtime, OS, middleware(WAS) = IaaS
    - [Software as a Service]
    - business logic, resource = PaaS
    - <cloud system>
    - multipoint 구조의 클럽 형태의 분산 처리 시스템
    - <distributed processing system>
    - 네트워크 상 노드들이 분산 처리를 분산 작업을 하고 이를 조화하는 시스템
    - <blockchain>
    - -P2P 구조의 비중앙 레이어 분산 장장 시스템
    - 트랜잭션 기록을 블록 단위로 묶어 체인 형태로 연결하고 각 블록은 이전 블록의 해시값을 포함하여 무결성을 보장하는 분산 원장 시스템. 작업증명 또는 지분증명 합의 알고리즘을 통해 모든 노드가 동일한 장부를 유지하도록 보장함.
    - 모든 노드가 동일한 장부를 유지하고 작업증명 또는 지분증명 합의 알고리즘을 통해 모든 노드가 동일한 장부를 유지하도록 보장함.
    - [이더리움] 가장 잘 인지되었던 사례 중 하나로 스마트 컨트랙트를 지원하는 분산 노드가 수 없이 생성을 담당함.
    - <HTTP method>
    - [GET] (URL로 리소스 조회
    - [POST]: (body에) 새로운 리소스 생성
    - [PUT]: (body에) 리소스 전체 교체, 없으면 생성
    - [PATCH]: (body에) 리소스 부분 교체
    - [DELETE] (URL로) 리소스 삭제
    - <URI(Uniform Resource Identifier)>
    - [resource]: server data
    - [URL(Uniform Resource Locator)]
    - -Scheme:[Host:Port]/Path?Query#Fragment
    - -path: 서버 내 리소스 위치 (필수), 서버에서 경로를 찾기 위한 주소
    - -Query/Param: body가 아닌 URL에 데이터를 넣어 전송
    - [URN(Uniform Resource Name)]
    - -이름처럼 일회
    - <request start-line>
    - URL로 자원한 서버에 요청
    - -HTTP-method: URL, HTTP-version
    - <status start-line>
    - -상태를 서버가 응답하는 방식
    - -HTTP-version State-code Reason-phrase
    - <state code>
    - [informational]
    - -100(Continue): 계속 요청 보냄
    - -Header(200): 헤 리소스를 클라이언트 캐싱 성공
    - <Redirection>: 요청한 URI에 대해 다른 URI를 리다이렉션
    - -Moved Permanently(301): 요청 URI가 이제 영구적으로 변경됨 (301 redirection
    - -Found(302): 요청 URI가 임시로 다른 URI에 있음 -redirection
    - -Not Modified(304): 브라우저에 저장된 캐시를 사용하라 -redirection
    - [client error]
    - -Unauthorized(401): 인증 필요
    - -Forbidden(403): 인증 했지만 권한 없음
    - -not found(404): 요청 리소스가 존재하지 않음
    - [Server error]
    - -Internal Server Error(500): 서버 내부 장애
    - -PRG(Post/Redirect/Get): POST 요청이 성공적 redirection되면 다시 요청을 보내서 결과를 보여주는 방식이다. 이 레이어를 새로운 요청을 하기 전에 한번 멈추어 있는 것
    - <HTTP header>
    - [key: value]
    - [Header name]
    - -Header value
    - <HTTP message>
    - <request header>
    - [host]: 요청하는 서버의 호스트 주소
    - [Referer]: 이전 페이지 URL
    - [User-Agent] Client가 사용 중인 OS, rendering engine, web-browser 정보
    - [Accept] Client가 선호하는 Content-Type
    - [Accept-Encoding] Client가 선호하는 Content-Encoding
    - [Accept-Language] Client가 선호하는 Content-Language
    - [Range]: 영역 전송 구간 지정
    - [Authorization]: 로그인 인증 정보
    - [Cookie]: 요청하는 cookie storage를 브라우저에 보내는 헤더
    - [If-Modified-Since]: 서버에게 물었던 리소스에 대하여 상태가 수정된 경우에만 body를 전송 (아니면 304 응답)
    - [If-None-Match]: 서버에게 물었던 리소스의 대하여 ETag가 수정된 경우에만 body를 전송 (아니면 304 응답)
    - <general header>
    - [Cache-Control]
    - -max age: 캐시된 리소스가 유효기간
    - -no-cache: origin server에 확인한 리소스가 유효한지 물어봄
    - -no-store: 캐시하지 않는 리소스 저장
    - -public: 모든 사용자가 공유 캐시에 저장 가능
    - -private: proxy server에 개인적인 캐시는 개인 데이터
    - <response header>
    - [Server]: origin 서버의 소프트웨어 정보
    - [Date]: 응답한 시간
    - [Location]: URI to redirect
    - [Content-Range]: 영역 전송한 범위 표시
    - [Transfer-Encoding]: body 전송 전송 방식 지정
    - [Content-Length] 사용하기
    - [WWW-Authenticate]: 401 응답 시 서버가 요구하는 인증 방식 정보
    - [Set-Cookie]
    - server가 client에 cookie를 저장하는 명령
    - [Cookie]: 요청할 때 client가 server에 되돌려 보내는 cookie
    - -domain: 쿠키를 적용하는 서버 도메인 지정
    - -path: domain의 상대 경로
    - -expires/max-age: 쿠키 만료일(expires)/남은 시간(max-age) 초과시 소멸
    - -secure: https에서만 쿠키 전송
    - -HttpOnly: 자바스크립트 접근 차단
    - [Last-Modified]: 캐시 유효성을 판단하는 마지막 변경일 표기
    - [ETag]: 특정 데이터의 내용이 변경되었는지 비교하는 고유 식별자
    - <request body type>
    - [x-www-form-urlencoded]
    - -key=value&key=value 형태
    - -body로 전송되기 때문에 특수 문자는 URL 인코딩 필요
    - [application/ison]
    - -API 통신에서
    - 가장 표준화된 데이터 포맷
    - <response body type>
    - [text/html]
    - .html, .htm, .jsp 등
    - [text/plain]
    - 일반 텍스트 전송
    - [application/ison]
    - -API 응답에서
    - 가장 표준화된 데이터 포맷
    - [기타 등등]
    - <HTTP Body>
    - [binary] 이미지, 파일 등
    - [byte] 최종 기술로 보낼 데이터
    - <encryption algorithm>
    - [symmetric key]
    - 암호화 키와 복호화 키 같음
    - [asymmetric key]
    - 암호화 키와 복호화 키 다름
    - <firewall>
    - traffic filtering
    - [<IPS(Intrusion Prevention System)]
    - -monitoring + blocking
    - [<IDS(Intrusion Detection System)]
    - -monitoring + warning
    - <network security>
    - [Confidentiality]
    - -encryption algorithm
    - -firewall
    - [Integrity]
    - -hash algorithm
    - -error control
    - [Availability]
    - -load balancing: distributed server
    - -failover: spare server
    - <Text Authentication>
    - [ID-password]
    - [physical ID]
    - <network Authentication>
    - <staying Authentication>
    - [session]
    - -서버에 로인 쉬세션을 저장하고 클라이언트는 세션 키만 보관
    - [Access/Token]
    - 클라이언트는 키만 보관하고 서버에 저장 -http1.1 stateless 이후로 인증정보를 요청할 때마다 제장
    - <Authentication>
    - <Authorization>
    - <Accounting>
    - <Encryption>
    - <Hash>
    - <Firewall>
    - <Proxy>
    - <Cache>
    - <Load Balancer>
    - <Backup>
    - <Monitoring>
    - <Log>
    - <Error Handling>
    - <Retry>
    - <Timeout>
    - <Rate Limiting>
    - <Queue>
    - <Cron>
    - <Scheduler>
    - <Worker>
    - <Middleware>
    - <Dependency Injection>
    - <Configuration>
    - <Environment Variable>
    - <Version>
    - <Deployment>
    - [build]
    - -소스 코드를 실행 가능한 결과물(바이너리)로 변환
    - [packaging]
    - -빌드된 코드와 의존성 파일들이 하나로 묶는 과정
    - [deployment]
    - -만들어 놓은 결과물을 실제 운영 서버에 전달하는 과정
    - <CI/CD>
    - [Continuous Integration]
    - 빌드, 테스트 자동화
    - [Continuous Deployment]
    - 배포 자동화
    - <continuous deployment>
    - 코드가 main 브랜치에 병합되면 자동으로 배포되는 방식
    - <connector>
    - hardware-hardware interface
    - <hardware>
    - <hardware>